



UNIVERSIDADE KIMPA VITA

Instituto Politécnico de Uíge

Licenciatura em Engenharia Informática · 3.º Ano

TRABALHO PRÁTICO DE SGBD II

Modelação e Implementação de um Sistema de e-Commerce

utilizando MongoDB — SGBD NoSQL

Por:

1. Américo Malungo Sebastião Miguel
2. Bengui Bena Pedro
3. Egas Manuel Ribeiro
4. Matondo Domingos Bunga
5. Hermenegildo Wilson dos Santos Panzo

DOCENTE

Moyo Kanivengidio

Uíge, Maio de 2026

Índice

1. Introdução	1
2. Objetivos do Trabalho.....	2
3. Enquadramento Teórico.....	3
3.1. Bases de Dados NoSQL.....	3
3.2. MongoDB	3
3.3. Modelo de Documentos	4
4. Modelação do Sistema	5
4.1. Visão Geral das Coleções	5
4.2. Estrutura das Coleções	5
4.2.1. Coleção: clientes	5
4.2.2. Coleção: produtos	6
4.2.3. Coleção: pedidos.....	7
4.2.4. Coleção: pagamentos	8
5. Implementação do CRUD.....	9
5.1. Create — Inserção de Documentos.....	9
5.2. Read — Consulta de Documentos	10
5.4. Delete — Remoção de Documentos	11
6. Arquitetura da Aplicação Python.....	11
7. Consultas de Agregação.....	13
7.1. Total de Vendas por Status de Pedido	13
7.2. Produtos Mais Vendidos.....	13
7.3. Receita Total por Método de Pagamento.....	13
Conclusão.....	14
Referências Bibliográficas	15

1. Introdução

O presente trabalho foi desenvolvido no âmbito da cadeira de Sistemas de Gestão de Bases de Dados II (SGBD II), do curso de Licenciatura em Informática da Universidade Kimpa Vita, sob orientação do Prof. Moyo Kanivengidio. O trabalho tem como finalidade a modelação e implementação de um sistema de e-Commerce utilizando um Sistema de Gestão de Bases de Dados NoSQL, nomeadamente o MongoDB.

O comércio eletrónico (e-Commerce) representa uma das aplicações mais relevantes dos sistemas de informação na atualidade, sendo responsável por transações de bens e serviços através de plataformas digitais. A gestão eficiente dos dados num sistema desta natureza exige uma infraestrutura de base de dados robusta, escalável e capaz de lidar com volumes variáveis de informação — características que os sistemas NoSQL, como o MongoDB, oferecem de forma nativa.

O MongoDB foi escolhido pelo grupo por ser um sistema orientado a documentos, de código aberto, que armazena dados em formato BSON (Binary JSON), permitindo uma modelação flexível e intuitiva que se adapta bem à natureza heterogénea dos dados de um sistema de e-Commerce.

2. Objetivos do Trabalho

Os objetivos gerais e específicos deste trabalho são:

- Compreender e aplicar os conceitos de bases de dados NoSQL orientadas a documentos.
- Modelar as coleções de um sistema de e-Commerce utilizando o MongoDB.
- Implementar as operações fundamentais de CRUD (Create, Read, Update, Delete) nas quatro entidades principais do sistema.
- Desenvolver uma aplicação em Python que integre o driver PyMongo para interação com o MongoDB.
- Elaborar scripts MongoDB Shell para criação de coleções, índices e dados de exemplo.
- Documentar todo o processo de modelação, implementação e testes realizados.

3. Enquadramento Teórico

3.1. Bases de Dados NoSQL

O termo NoSQL (Not Only SQL) refere-se a uma classe de sistemas de gestão de bases de dados que não seguem exclusivamente o modelo relacional tradicional. Estes sistemas emergiram para dar resposta às limitações dos SGBD relacionais perante os requisitos modernos de escalabilidade, flexibilidade e desempenho em grandes volumes de dados (Big Data).

Os sistemas NoSQL classificam-se em quatro grandes categorias: orientados a documentos (MongoDB, CouchDB), baseados em chave-valor (Redis, DynamoDB), baseados em colunas (Cassandra, HBase) e baseados em grafos (Neo4j, ArangoDB). Cada categoria é otimizada para casos de uso específicos.

3.2. MongoDB

O MongoDB é um sistema de gestão de bases de dados NoSQL orientado a documentos, desenvolvido pela MongoDB, Inc., lançado em 2009. É atualmente um dos SGBD NoSQL mais utilizados a nível mundial, com suporte para múltiplas plataformas.

As principais características do MongoDB são:

- Armazenamento de dados em documentos BSON (Binary JSON), flexíveis e autodescritivos.
- Ausência de esquema rígido (schema-less), permitindo documentos com estruturas distintas na mesma coleção.
- Suporte nativo para escalabilidade horizontal através de sharding.
- Replicação automática para alta disponibilidade através de replica sets.
- Linguagem de consulta poderosa, incluindo o pipeline de agregação.
- Índices secundários para otimização de consultas.

3.3. Modelo de Documentos

No MongoDB, os dados são organizados em coleções, equivalentes conceitualmente às tabelas dos SGBD relacionais. Cada coleção contém documentos, que são análogos às linhas/tuplos, mas com a diferença fundamental de poderem conter campos aninhados e arrays, eliminando a necessidade de joins complexos.

Um documento MongoDB é representado em formato JSON/BSON, onde cada campo possui um nome e um valor. O campo `_id` é obrigatório e serve como chave primária única, sendo gerado automaticamente pelo MongoDB através do tipo `ObjectId` se não for fornecido.

A grande vantagem do modelo de documentos para um sistema de e-Commerce reside na possibilidade de embutir (embed) documentos relacionados num único documento pai — como os itens de um pedido embutidos diretamente no documento do pedido — reduzindo a necessidade de múltiplas consultas.

4. Modelação do Sistema

4.1. Visão Geral das Coleções

O sistema de e-Commerce modelado é composto por quatro coleções principais, que representam as entidades fundamentais do domínio de negócio: Clientes, Produtos, Pedidos e Pagamentos. As relações entre estas coleções são mantidas através de referências por ID (padrão de referência), com exceção dos itens do pedido, que são embutidos diretamente no documento de pedido (padrão de embedding).

4.2. Estrutura das Coleções

4.2.1. Coleção: clientes

Armazena a informação dos clientes registados na plataforma.

Campo	Tipo BSON	Descrição
_id	ObjectId	Identificador único gerado automaticamente
Nome	String	Nome completo do cliente
Email	String	Endereço de email (campo único — índice único)
Telefone	String	Número de contacto telefónico
Endereço	String	Morada de entrega predefinida
data_registo	Date	Data e hora do registo na plataforma
Ativo	Boolean	Estado lógico (soft delete)

4.2.2. Coleção: produtos

Armazena o catálogo de produtos disponíveis para compra.

Campo	Tipo BSON	Descrição
_id	ObjectId	Identificador único gerado automaticamente
Nome	String	Nome comercial do produto
Descrição	String	Descrição detalhada do produto
Preco	Double	Preço de venda em Kwanzas (AOA)
Categoria	String	Categoria do produto (índice secundário)
Stock	Int32	Quantidade disponível em armazém
data_criacao	Date	Data de adição ao catálogo
Ativo	Boolean	Disponibilidade do produto (soft delete)

4.2.3. Coleção: pedidos

Regista os pedidos efetuados pelos clientes. Os itens do pedido são embutidos (embedded) diretamente no documento do pedido, evitando múltiplas consultas.

Campo	Tipo BSON	Descrição
_id	ObjectId	Identificador único do pedido
cliente_id	String	Referência ao _id do cliente
Itens	Array	Lista de itens: {produto_id, nome, preco_unitario, quantidade}
Total	Double	Valor total calculado do pedido (em AOA)
Status	String	Estado: pendente processando enviado entregue cancelado
data_pedido	Date	Data e hora em que o pedido foi criado
data_atualizacao	Date	Data da última atualização do pedido

4.2.4. Coleção: pagamentos

Regista os pagamentos associados a cada pedido. Existe uma relação de um-para-um com a coleção de pedidos.

Campo	Tipo BSON	Descrição
_id	ObjectId	Identificador único do pagamento
pedido_id	String	Referência ao _id do pedido (índice único)
Valor	Double	Valor total pago (em AOA)
Método	String	Método: dinheiro transferência multicaixa cartão
Status	String	Estado: pendente confirmado falhado reembolsado
data_pagamento	Date	Data e hora do registo do pagamento
data_atualizacao	Date	Data da última atualização

5. Implementação do CRUD

As operações CRUD foram implementadas em dois contextos complementares: (1) através de scripts MongoDB Shell (.js) para demonstração das operações nativas do MongoDB e (2) através de módulos Python que utilizam o driver PyMongo para integração numa aplicação interativa.

Coleção	Create	Read	Update	Delete
clientes	insertOne()	find() / findOne()	updateOne()	updateOne() ativo:false
produtos	insertOne()	find() / findOne()	updateOne()	updateOne() ativo:false
pedidos	insertOne()	find() / findOne()	updateOne()	deleteOne()
pagamentos	insertOne()	find() / findOne()	updateOne()	deleteOne()

5.1. Create — Inserção de Documentos

A operação de inserção utiliza o método insertOne() para adicionar um único documento a uma coleção. O MongoDB gera automaticamente o campo _id com um ObjectId único se não for fornecido.

Exemplo — Inserção de um cliente no MongoDB Shell:

```
db.clientes.insertOne({
  nome: "Ana Paula Lopes",
  email: "ana.lopes@gmail.com",
  telefone: "+244 923 456 789",
  endereco: "Rua da Liberdade, 12, Uíge",
  data_registro: new Date(),
  ativo: true
});
```

No módulo Python (clientes.py), a mesma operação é realizada através do driver PyMongo:

```
def criar(self, nome, email, telefone, endereco):
    documento = { "nome": nome, "email": email,
                  "telefone": telefone, "endereco": endereco,
                  "data_registro": datetime.now(), "ativo": True }
    resultado = self.col.insert_one(documento)
    return str(resultado.inserted_id)
```

5.2. Read — Consulta de Documentos

As operações de leitura utilizam os métodos find() para retornar múltiplos documentos e findOne() para retornar um único documento. É possível aplicar filtros, projeções e ordenação.

Exemplo — Consultar todos os clientes ativos:

```
db.clientes.find({ ativo: true });
```

Exemplo — Consultar produtos de uma categoria específica com projeção:

```
db.produtos.find(
    { categoria: "Electrónica", ativo: true },
    { nome: 1, preco: 1, stock: 1, _id: 0 }
);
```

Exemplo — Consultar pedidos de um cliente, ordenados por data decrescente:

```
db.pedidos.find({ cliente_id: "ID_DO_CLIENTE" })
    .sort({ data_pedido: -1 });
```

5.3. Update — Atualização de Documentos

As operações de atualização utilizam updateOne() para modificar um documento ou updateMany() para modificar múltiplos documentos. O operador \$set define os campos a atualizar e o operador \$inc realiza incrementos/decrementos.

Exemplo — Atualizar o status de um pedido:

```
db.pedidos.updateOne(
    { _id: ObjectId("ID_DO_PEDIDO") },
    { $set: { status: "enviado",
              data_atualizacao: new Date() } }
);
```

Exemplo — Decrementar o stock de um produto após venda:

```
db.produtos.updateOne(  
  { _id: ObjectId("ID_DO_PRODUTO"), stock: { $gte: 1 } },  
  { $inc: { stock: -1 } }  
);
```

Exemplo — Desativar em massa todos os produtos sem stock:

```
db.produtos.updateMany(  
  { stock: 0 },  
  { $set: { ativo: false } }  
);
```

5.4. Delete — Remoção de Documentos

O sistema implementa dois padrões de remoção: o soft delete (remoção lógica), que mantém o documento na coleção mas marca o campo ativo como false — utilizado em clientes e produtos para preservar o histórico — e o hard delete (remoção física), utilizado em pedidos e pagamentos através do método deleteOne().

Exemplo — Soft delete de um cliente:

```
db.clientes.updateOne(  
  { _id: ObjectId("ID_DO_CLIENTE") },  
  { $set: { ativo: false,  
            data Eliminacao: new Date() } }  
);
```

Exemplo — Hard delete de um pagamento:

```
db.pagamentos.deleteOne(  
  { _id: ObjectId("ID_DO_PAGAMENTO") }  
);
```

6. Arquitetura da Aplicação Python

A aplicação desenvolvida em Python segue uma arquitetura modular, onde cada coleção do MongoDB possui um módulo dedicado com as respectivas classes e métodos CRUD. Esta organização promove a separação de responsabilidades, a reutilização de código e a facilidade de manutenção.

Ficheiro	Responsabilidade
database.py	Ligação ao MongoDB (MongoClient). Retorna a instância da base de dados.
clientes.py	Classe ClientesCRUD com métodos: criar, listar_todos, buscar_por_email, buscar_por_id, atualizar, eliminar.
produtos.py	Classe ProdutosCRUD com métodos: criar, listar_todos, buscar_por_categoria, buscar_por_id, atualizar, decrementar_stock, eliminar.
pedidos.py	Classe PedidosCRUD com métodos: criar, listar_todos, buscar_por_cliente, buscar_por_id, atualizar_status, eliminar.
pagamentos.py	Classe PagamentosCRUD com métodos: criar, listar_todos, buscar_por_pedido, buscar_por_id, atualizar_status, eliminar.
main.py	Ponto de entrada da aplicação. Menus interativos para operações CRUD em todas as coleções.

A comunicação com o MongoDB é realizada através da biblioteca PyMongo (versão 4.7.2), que disponibiliza uma API Python completa e idiomática para todas as operações do MongoDB. A ligação é gerida pelo módulo database.py, que inicializa o MongoClient e verifica a disponibilidade do servidor antes de retornar a instância da base de dados.

Para execução da aplicação, é necessário ter o MongoDB instalado e em execução na máquina local (porta 27017), instalar as dependências Python listadas no ficheiro requirements.txt, executar o script de configuração setup.js no MongoDB Shell para criar as coleções e índices, e finalmente executar o ficheiro main.py.

7. Consultas de Agregação

Para além das operações CRUD básicas, o sistema implementa consultas de agregação utilizando o pipeline de agregação do MongoDB. Este mecanismo permite realizar operações de agrupamento, filtragem, ordenação e cálculos sobre os documentos das coleções.

7.1. Total de Vendas por Status de Pedido

Esta consulta agrupa os pedidos pelo campo status, contabilizando o número de pedidos e o valor total em cada estado:

```
db.pedidos.aggregate([
  { $group: { _id: "$status",
              total_pedidos: { $sum: 1 },
              valor_total: { $sum: "$total" } } },
  { $sort: { total_pedidos: -1 } }
]);
```

7.2. Produtos Mais Vendidos

Esta consulta utiliza o estágio \$unwind para desdobrar o array de itens de cada pedido e depois agrupa por produto para calcular o total de unidades vendidas:

```
db.pedidos.aggregate([
  { $unwind: "$itens" },
  { $group: { _id: "$itens.nome",
              total_vendido: { $sum: "$itens.quantidade" } } },
  { $sort: { total_vendido: -1 } },
  { $limit: 5 }
]);
```

7.3. Receita Total por Método de Pagamento

Esta consulta agrupa os pagamentos confirmados pelo método utilizado:

```
db.pagamentos.aggregate([
  { $match: { status: "confirmado" } },
  { $group: { _id: "$metodo",
              receita: { $sum: "$valor" },
              n_pagamentos: { $sum: 1 } } },
  { $sort: { receita: -1 } }
]);
```

Conclusão

O presente trabalho permitiu ao grupo adquirir e consolidar conhecimentos práticos sobre a modelação e implementação de bases de dados NoSQL, com enfoque no MongoDB. Através do desenvolvimento do sistema de e-Commerce, foi possível compreender as diferenças fundamentais entre o paradigma relacional e o paradigma orientado a documentos, bem como as vantagens e limitações de cada abordagem.

A flexibilidade do modelo de documentos do MongoDB revelou-se particularmente vantajosa na representação dos itens dos pedidos, que foram embutidos diretamente no documento do pedido, eliminando a necessidade de operações de join e simplificando as consultas mais frequentes do sistema.

A implementação das operações CRUD em dois contextos — MongoDB Shell e Python/PyMongo — demonstrou a versatilidade do MongoDB e a facilidade de integração com linguagens de programação de propósito geral. O padrão de soft delete adotado para clientes e produtos, em detrimento da remoção física dos documentos, garante a integridade do histórico de transações, o que é fundamental num contexto de e-Commerce.

Como trabalhos futuros, o grupo considera a implementação de autenticação de utilizadores, a adição de coleções para categorias e avaliações de produtos, a integração com APIs de pagamento, e o desenvolvimento de uma interface gráfica web utilizando Flask ou FastAPI.

Referências Bibliográficas

- Chodorow, K. (2013). MongoDB: The Definitive Guide (2.^a ed.). O'Reilly Media.
- Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7.^a ed.). Pearson.
- Fowler, M., & Sadalage, P. J. (2012). NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Addison-Wesley.
- MongoDB, Inc. (2024). MongoDB Manual (v7.0). <https://www.mongodb.com/docs/manual/>
- MongoDB, Inc. (2024). PyMongo Documentation (v4.7). <https://pymongo.readthedocs.io/>
- Python Software Foundation. (2024). Python 3 Documentation. <https://docs.python.org/3/>
- Anthropic. (2026). *Claude* (Versão 4.6 Sonnet) [Modelo de linguagem grande]. <https://claude.ai>